OOP: Polymorphism and Interfaces

Visual C# How to Program, 6/e

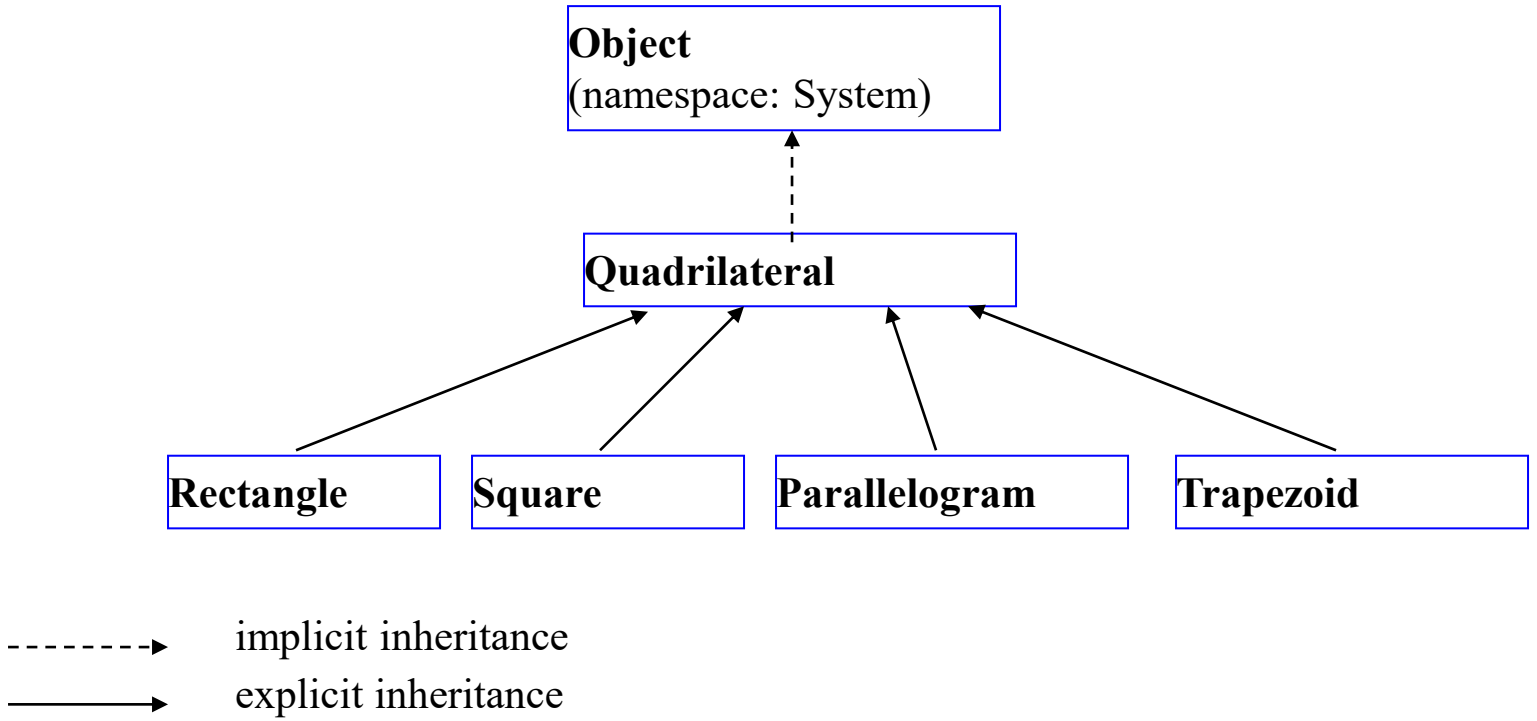
Introduction

- **Polymorphism** enables you to write apps that process objects that share the same base class in a class hierarchy as if they were all objects of the base class.
- Polymorphism promotes extensibility.

Polymorphism Examples

- If class `Rectangle` is derived from class `Quadrilateral`, then a `Rectangle` is a more specific version of a `Quadrilateral`.
- Any operation that can be performed on a `Quadrilateral` object can also be performed on a `Rectangle` object.
- These operations also can be performed on other `Quadrilaterals`, such as `Squares`, `Parallelograms` and `Trapezoids`.
- The polymorphism occurs when an app invokes a method through a base-class variable.

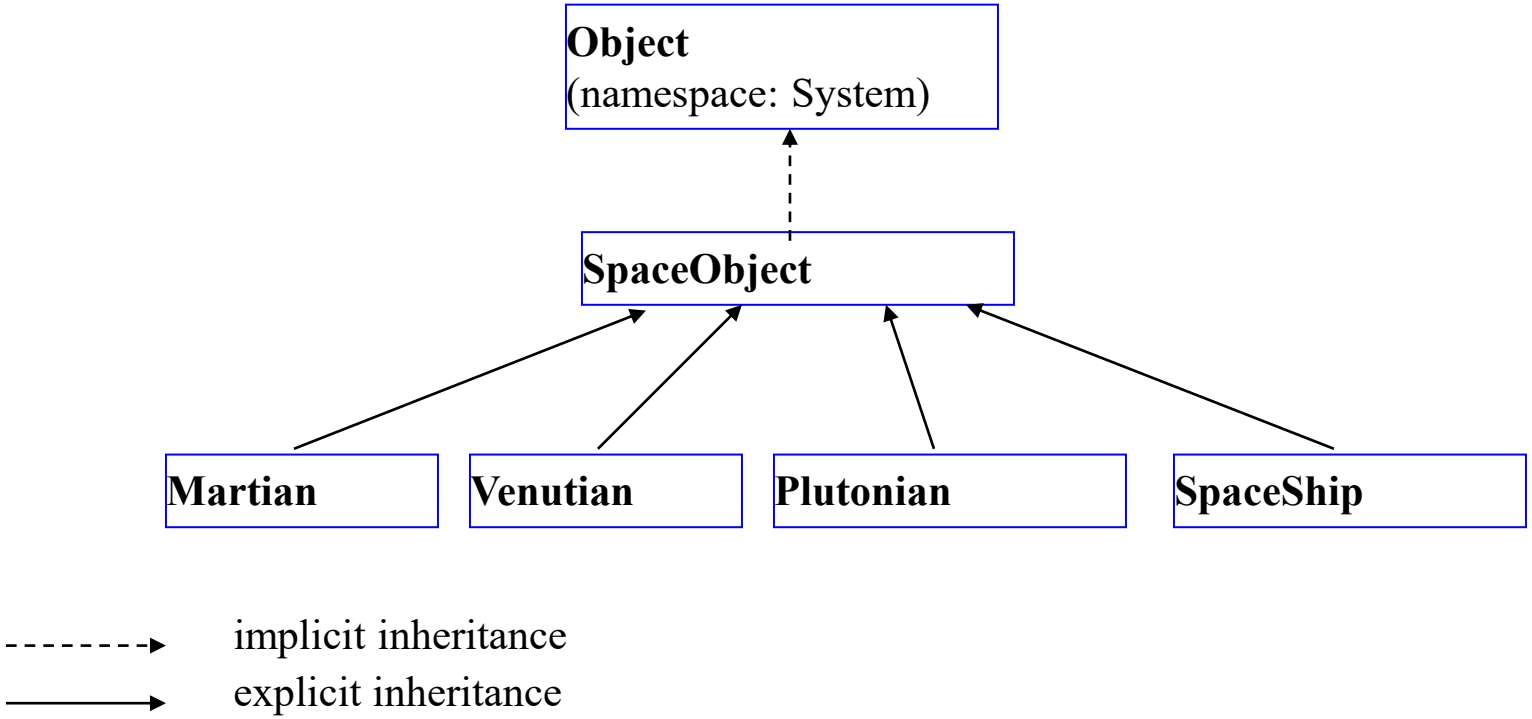
Polymorphism Examples



Polymorphism Examples (Cont.)

- As another example, suppose we design a video game that manipulates objects of many different types, including objects of classes `Martian`, `Venusian`, `Plutonian`, `SpaceShip` and `LaserBeam`.
- Each class inherits from the common base class `SpaceObject`, which contains method `Draw`.
- A screen-manager app maintains a collection (e.g., a `SpaceObject` array) of references to objects of the various classes.
- To refresh the screen, the screen manager periodically sends each object the same message—namely, `Draw`, while object responds in a unique way.

Polymorphism Examples (Cont.)



- A screen-manager program may contain a **SpaceObject** array of references to objects of various classes that derive from **SpaceObject**
  - ```
    arrOfSpaceObjects[ 0 ] = martian1;
    arrOfSpaceObjects[ 1 ] = martian2;
    arrOfSpaceObjects[ 2 ] = venutian1;
    arrOfSpaceObjects[ 3 ] = plutonian1;
    arrOfSpaceObjects[ 4 ] = spaceShip1;
    ```
    …
- To refresh the screen, the screen-manager calls **DrawYourself** on each object in the array
  - ```
    foreach(SpaceObject spaceObject in arrOfSpaceObjects)
    {
       spaceObject.DrawYourself
    }
    ```
- The program **polymorphically** calls the appropriate version of **DrawYourself** on each object, based on the type of that object

Abstract Classes and Methods

- Abstract classes
  - Cannot be instantiated
  - Used as base classes
  - Class definitions are not complete
    - Derived classes must define the missing pieces
  - Can contain abstract methods and/or abstract properties
    - Have no implementation
    - Derived classes must override inherited abstract methods and abstract properties to enable instantiation

      Abstract methods and abstract properties are implicitly virtual

Abstract Classes and Methods (Cont.)

- An abstract class is used to provide an appropriate base class from which other classes may inherit (concrete classes)
- Abstract base classes are too generic to define (by instantiation) real objects
- To define an abstract class, use keyword **abstract** in the declaration
- To declare a method or property abstract, use keyword **abstract** in the declaration
- Abstract methods and properties have no implementation

Abstract Classes and Methods (Cont.)

- Concrete classes use the keyword **override** to provide implementations for all the abstract methods and properties of the base-class
- Any class with an abstract method or property must be declared **abstract**
- Even though abstract classes cannot be instantiated, we can use abstract class references to refer to instances of any concrete class derived from the abstract class
- Constructors and static methods cannot be declared `abstract` or `virtual`

Abstract Classes and Methods (Cont.)

- Abstract property declarations have the form:
  - `public abstract PropertyType MyProperty { get; set; }`
- An `abstract` property omits implementations for the `get` accessor and/or the `set` accessor.
- Concrete derived classes must provide implementations for every accessor declared in the `abstract` property.

Case Study: Payroll System Using Polymorphism

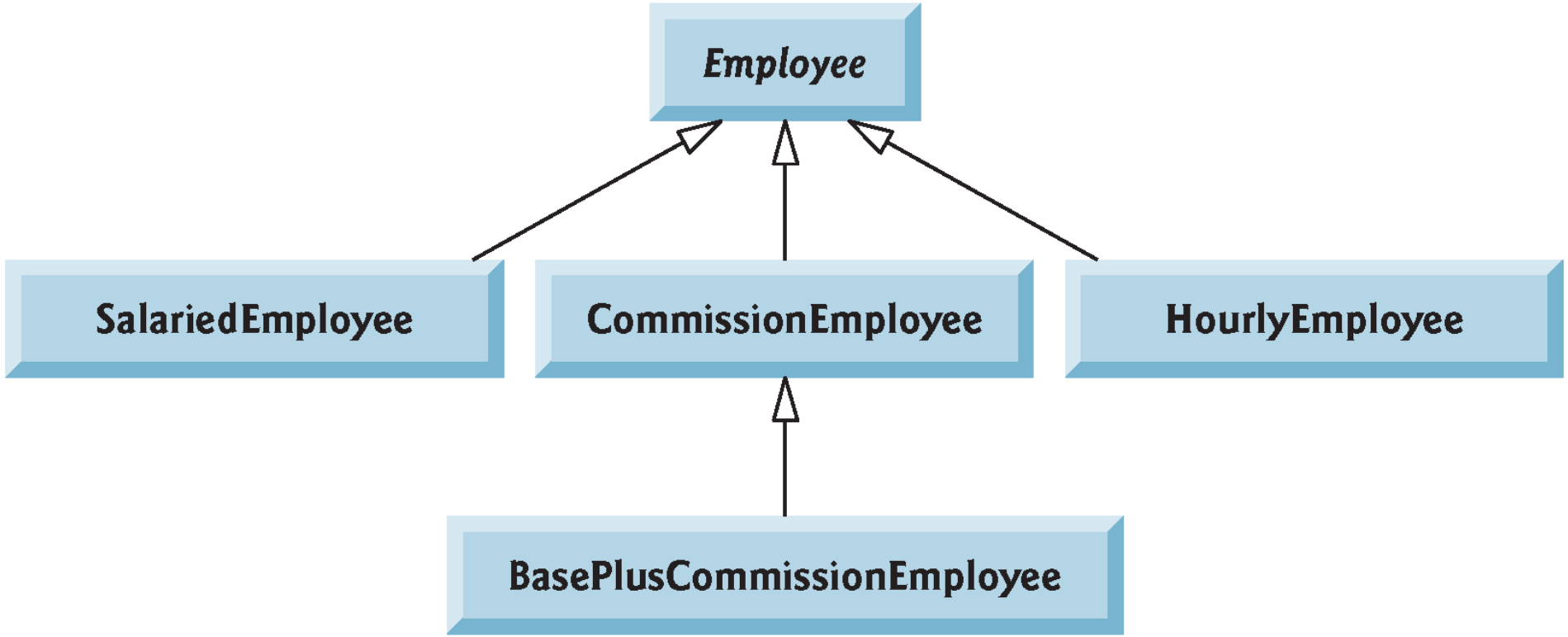
- A company pays its employees on a weekly basis. The employees are of four types:
  - Salaried employees are paid a fixed weekly salary regardless of the number of hours worked,
  - hourly employees are paid by the hour and receive "time-and-a-half" overtime pay for all hours worked in excess of 40 hours,
  - commission employees are paid a percentage of their sales, and
  - salaried-commission employees receive a base salary plus a percentage of their sales.
- For the current pay period, the company has decided to reward salaried-commission employees by adding 10% to their base salaries. The company wants to implement an app that performs its payroll calculations polymorphically.

Case Study: Payroll System Using Polymorphism

- We use `abstract` class `Employee` to represent the general concept of an employee.
- `SalariedEmployee`, `CommissionEmployee` and `HourlyEmployee` extend `Employee`.
- Class `BasePlusCommissionEmployee`—which extends `CommissionEmployee`—represents the last employee type.

Case Study: Payroll System Using Polymorphism

- The UML class diagram shows the inheritance hierarchy for our polymorphic employee payroll app.



Case Study: Payroll System Using Polymorphism (Cont.)

- The diagram shows each of the five classes in the hierarchy down the left side and methods `Earnings` and `ToString` across the top.

	Earnings	ToString
Employee	abstract	*firstName lastName* social security number: *SSN*
Salaried-Employee	weeklySalary	salaried employee: *firstName lastName* social security number: *SSN* weekly salary: *weeklysalary*
Hourly-Employee	*If hours <= 40* wage * hours *If hours > 40* 40 * wage + (hours - 40) * wage * 1.5	hourly employee: *firstName lastName* social security number: *SSN* hourly wage: *wage* hours worked: *hours*
Commission-Employee	commissionRate * grossSales	commission employee: *firstName lastName* social security number: *SSN* gross sales: *grossSales* commission rate: *commissionRate*
BasePlus-Commission-Employee	baseSalary + (commissionRate * grossSales)	base salaried commission employee: *firstName lastName* social security number: *SSN* gross sales: *grossSales* commission rate: *commissionRate* base salary: *baseSalary*

Creating Abstract Base Class Employee

- Class `Employee` provides methods `Earnings` and `ToString`, in addition to the properties that manipulate `Employee`'s instance variables.
- Each earnings calculation depends on the employee's class, so we declare `Earnings` as `abstract`.
- The app iterates through the array and calls method `Earnings` for each `Employee` object. These method calls are processed polymorphically.
- Each derived class overrides method `ToString` to create a `string` representation of an object of that class.

```
1  // Fig. 12.4: Employee.cs
2  // Employee abstract base class.
3  public abstract class Employee
4  {
5     public string FirstName { get; }
6     public string LastName { get; }
7     public string SocialSecurityNumber { get; }
8
9     // three-parameter constructor
10    public Employee(string firstName, string lastName,
11       string socialSecurityNumber)
12    {
13       FirstName = firstName;
14       LastName = lastName;
15       SocialSecurityNumber = socialSecurityNumber;
16    }
17
18    // return string representation of Employee object, using properties
19    public override string ToString() => $"{FirstName} {LastName}\n" +
20       $"social security number: {SocialSecurityNumber}";
21
22    // abstract method overridden by derived classes
23    public abstract decimal Earnings(); // no implementation here
24 }
```

```
1  // Fig. 12.5: SalariedEmployee.cs
2  // SalariedEmployee class that extends Employee.
3  using System;
4
5  public class SalariedEmployee : Employee
6  {
7     private decimal weeklySalary;
8
9     // four-parameter constructor
10    public SalariedEmployee(string firstName, string lastName,
11       string socialSecurityNumber, decimal weeklySalary)
12       : base(firstName, lastName, socialSecurityNumber)
13    {
14       WeeklySalary = weeklySalary; // validate salary
15    }
16
```

```
17      // property that gets and sets salaried employee's salary
18      public decimal WeeklySalary
19      {
20         get
21         {
22            return weeklySalary;
23         }
24         set
25         {
26            if (value < 0) // validation
27            {
28               throw new ArgumentOutOfRangeException(nameof(value),
29                  value, $"{nameof(WeeklySalary)} must be >= 0");
30            }
31
32            weeklySalary = value;
33         }
34      }
```

```
35
36      // calculate earnings; override abstract method Earnings in Employee
37      public override decimal Earnings() => WeeklySalary;
38
39      // return string representation of SalariedEmployee object
40      public override string ToString() =>
41         $"salaried employee: {base.ToString()}\n" +
42         $"weekly salary: {WeeklySalary:C}";
43   }
```

```
1  // Fig. 12.6: HourlyEmployee.cs
2  // HourlyEmployee class that extends Employee.
3  using System;
4
5  public class HourlyEmployee : Employee
6  {
7     private decimal wage; // wage per hour
8     private decimal hours; // hours worked for the week
9
10    // five-parameter constructor
11    public HourlyEmployee(string firstName, string lastName,
12       string socialSecurityNumber, decimal hourlyWage,
13       decimal hoursWorked)
14       : base(firstName, lastName, socialSecurityNumber)
15    {
16       Wage = hourlyWage; // validate hourly wage
17       Hours = hoursWorked; // validate hours worked
18    }
19
```

```
20      // property that gets and sets hourly employee's wage
21      public decimal Wage
22      {
23         get
24         {
25            return wage;
26         }
27         set
28         {
29            if (value < 0) // validation
30            {
31               throw new ArgumentOutOfRangeException(nameof(value),
32                  value, $"{nameof(Wage)} must be >= 0");
33            }
34
35            wage = value;
36         }
37      }
38
```

```
39      // property that gets and sets hourly employee's hours
40      public decimal Hours
41      {
42         get
43         {
44            return hours;
45         }
46         set
47         {
48            if (value < 0 || value > 168) // validation
49            {
50               throw new ArgumentOutOfRangeException(nameof(value),
51                  value, $"{nameof(Hours)} must be >= 0 and <= 168");
52            }
53
54            hours = value;
55         }
56      }
57
```

```
58      // calculate earnings; override Employee's abstract method Earnings
59      public override decimal Earnings()
60      {
61         if (Hours <= 40) // no overtime
62         {
63            return Wage * Hours;
64         }
65         else
66         {
67            return (40 * Wage) + ((Hours - 40) * Wage * 1.5M);
68         }
69      }
70
71      // return string representation of HourlyEmployee object
72      public override string ToString() =>
73         $"hourly employee: {base.ToString()}\n" +
74         $"hourly wage: {Wage:C}\nhours worked: {Hours:F2}";
75   }
```

```
1  // Fig. 12.7: CommissionEmployee.cs
2  // CommissionEmployee class that extends Employee.
3  using System;
4
5  public class CommissionEmployee : Employee
6  {
7     private decimal grossSales; // gross weekly sales
8     private decimal commissionRate; // commission percentage
9
10    // five-parameter constructor
11    public CommissionEmployee(string firstName, string lastName,
12       string socialSecurityNumber, decimal grossSales,
13       decimal commissionRate)
14       : base(firstName, lastName, socialSecurityNumber)
15    {
16       GrossSales = grossSales; // validates gross sales
17       CommissionRate = commissionRate; // validates commission rate
18    }
19
```

```
20      // property that gets and sets commission employee's gross sales
21      public decimal GrossSales
22      {
23         get
24         {
25            return grossSales;
26         }
27         set
28         {
29            if (value < 0) // validation
30            {
31               throw new ArgumentOutOfRangeException(nameof(value),
32                  value, $"{nameof(GrossSales)} must be >= 0");
33            }
34
35            grossSales = value;
36         }
37      }
38
```

```
39      // property that gets and sets commission employee's commission rate
40      public decimal CommissionRate
41      {
42         get
43         {
44            return commissionRate;
45         }
46         set
47         {
48            if (value <= 0 || value >= 1) // validation
49            {
50               throw new ArgumentOutOfRangeException(nameof(value),
51                  value, $"{nameof(CommissionRate)} must be > 0 and < 1");
52            }
53
54            commissionRate = value;
55         }
56      }
57
```

```
58      // calculate earnings; override abstract method Earnings in Employee
59      public override decimal Earnings() => CommissionRate * GrossSales;
60
61      // return string representation of CommissionEmployee object
62      public override string ToString() =>
63         $"commission employee: {base.ToString()}\n" +
64         $"gross sales: {GrossSales:C}\n" +
65         $"commission rate: {CommissionRate:F2}";
66   }
```

```
1  // Fig. 12.8: BasePlusCommissionEmployee.cs
2  // BasePlusCommissionEmployee class that extends CommissionEmployee.
3  using System;
4
5  public class BasePlusCommissionEmployee : CommissionEmployee
6  {
7     private decimal baseSalary; // base salary per week
8
9     // six-parameter constructor
10    public BasePlusCommissionEmployee(string firstName, string lastName,
11       string socialSecurityNumber, decimal grossSales,
12       decimal commissionRate, decimal baseSalary)
13       : base(firstName, lastName, socialSecurityNumber,
14          grossSales, commissionRate)
15    {
16       BaseSalary = baseSalary; // validates base salary
17    }
18
```

```
19      // property that gets and sets
20      // BasePlusCommissionEmployee's base salary
21      public decimal BaseSalary
22      {
23         get
24         {
25            return baseSalary;
26         }
27         set
28         {
29            if (value < 0) // validation
30            {
31               throw new ArgumentOutOfRangeException(nameof(value),
32                  value, $"{nameof(BaseSalary)} must be >= 0");
33            }
34
35            baseSalary = value;
36         }
37      }
38
```

```
39      // calculate earnings
40      public override decimal Earnings() => BaseSalary + base.Earnings();
41
42      // return string representation of BasePlusCommissionEmployee
43      public override string ToString() =>
44         $"base-salaried {base.ToString()}\nbase salary: {BaseSalary:C}";
45   }
```

Polymorphic Processing, Operator is and Downcasting

- The app tests our `Employee` hierarchy.

```
1  // Fig. 12.9: PayrollSystemTest.cs
2  // Employee hierarchy test app.
3  using System;
4  using System.Collections.Generic;
5
6  class PayrollSystemTest
7  {
8     static void Main()
9     {
10       // create derived-class objects
11       var salariedEmployee = new SalariedEmployee("John", "Smith",
12          "111-11-1111", 800.00M);
13       var hourlyEmployee = new HourlyEmployee("Karen", "Price",
14          "222-22-2222", 16.75M, 40.0M);
15       var commissionEmployee = new CommissionEmployee("Sue", "Jones",
16          "333-33-3333", 10000.00M, .06M);
17       var basePlusCommissionEmployee =
18          new BasePlusCommissionEmployee("Bob", "Lewis",
19          "444-44-4444", 5000.00M, .04M, 300.00M);
```

```
20
21          Console.WriteLine("Employees processed individually:\n");
22
23          Console.WriteLine($"{salariedEmployee}\nearned: " +
24              $"{salariedEmployee.Earnings():C}\n");
25          Console.WriteLine(
26             $"{hourlyEmployee}\nearned: {hourlyEmployee.Earnings():C}\n");
27          Console.WriteLine($"{commissionEmployee}\nearned: " +
28              $"{commissionEmployee.Earnings():C}\n");
29          Console.WriteLine($"{basePlusCommissionEmployee}\nearned: " +
30              $"{basePlusCommissionEmployee.Earnings():C}\n");
31
32          // create List<Employee> and initialize with employee objects
33          var employees = new List<Employee>() {salariedEmployee,
34             hourlyEmployee, commissionEmployee, basePlusCommissionEmployee};
35
```

```
36          Console.WriteLine("Employees processed polymorphically:\n");
37
38          // generically process each element in employees
39          foreach (var currentEmployee in employees)
40          {
41             Console.WriteLine(currentEmployee); // invokes ToString
42
43             // determine whether element is a BasePlusCommissionEmployee
44             if (currentEmployee is BasePlusCommissionEmployee)
45             {
46                // downcast Employee reference to
47                // BasePlusCommissionEmployee reference
48                var employee = (BasePlusCommissionEmployee) currentEmployee;
49
50                employee.BaseSalary *= 1.10M;
51                Console.WriteLine("new base salary with 10% increase is: " +
52                   $"{employee.BaseSalary:C}");
53             }
54
55             Console.WriteLine($"earned: {currentEmployee.Earnings():C}\n");
56          }
```

```
57
58          // get type name of each object in employees
59          for (int j = 0; j < employees.Count; j++)
60          {
61             Console.WriteLine(
62                $"Employee {j} is a {employees[j].GetType()}");
63          }
64       }
65    }
```

```
Employees processed individually:

salaried employee: John Smith
social security number: 111-11-1111
weekly salary: $800.00
earned: $800.00

hourly employee: Karen Price
social security number: 222-22-2222
hourly wage: $16.75
hours worked: 40.00
earned: $670.00

commission employee: Sue Jones
social security number: 333-33-3333
gross sales: $10,000.00
commission rate: 0.06
earned: $600.00

base-salaried commission employee: Bob Lewis
social security number: 444-44-4444
gross sales: $5,000.00
commission rate: 0.04
base salary: $300.00
earned: $500.00
```

```
Employees processed polymorphically:

salaried employee: John Smith
social security number: 111-11-1111
weekly salary: $800.00
earned: $800.00

hourly employee: Karen Price
social security number: 222-22-2222
hourly wage: $16.75
hours worked: 40.00
earned: $670.00

commission employee: Sue Jones
social security number: 333-33-3333
gross sales: $10,000.00
commission rate: 0.06
earned: $600.00

base-salaried commission employee: Bob Lewis
social security number: 444-44-4444
gross sales: $5,000.00
commission rate: 0.04
base salary: $300.00
new base salary with 10% increase is: $330.00
earned: $530.00
```

```
Employee 0 is a SalariedEmployee
Employee 1 is a HourlyEmployee
Employee 2 is a CommissionEmployee
Employee 3 is a BasePlusCommissionEmployee
```

Polymorphic Processing, Operator `is` and Downcasting (Cont.)

- You can avoid a potential `InvalidCastException` by using the as operator to perform a downcast rather than a cast operator.
  - If the downcast is invalid, the expression will be null instead of throwing an exception.
- Method `GetType` returns an object of class `Type` (of namespace `System`), which contains information about the object's type, including its class name, the names of its methods, and the name of its base class.
- The `Type` class's `ToString` method returns the class name.

Summary of the Allowed Assignments Between Base-Class and Derived-Class Variables

- Assigning a base-class reference to a base-class variable is straightforward.
- Assigning a derived-class reference to a derived-class variable is straightforward.
- Assigning a derived-class reference to a base-class variable is safe, because the derived-class object *is an* object of its base class. However, this reference can be used to refer only to base-class members.
- Attempting to assign a base-class reference to a derived-class variable is a compilation error. To avoid this error, the base-class reference must be cast to a derived-class type explicitly.

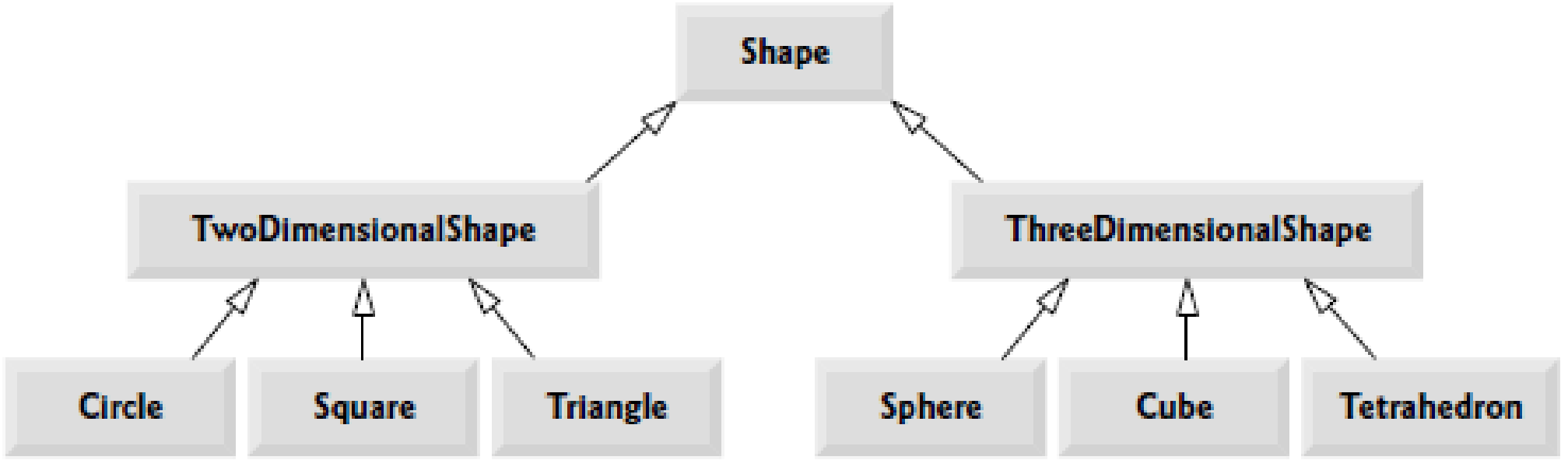
`sealed` Methods and Classes

- A method declared sealed in a base class cannot be overridden in a derived class.
- Methods that are declared `private` are implicitly `sealed`.
- Methods that are declared `static` also are implicitly `sealed`, because `static` methods cannot be overridden either.
- A derived-class method declared both `override` and `sealed` can override a base-class method, but cannot be overridden in classes further down the inheritance hierarchy.
- Calls to `sealed` methods (and non-`virtual` methods) are resolved at compile time—this is known as static binding.

sealed Methods and Classes (Cont.)

- A class that is declared `sealed` cannot be a base class (i.e., a class cannot extend a `sealed` class).
- All methods in a `sealed` class are implicitly `sealed`.
- Class `string` is a `sealed` class. This class cannot be extended, so apps that use `strings` can rely on the functionality of `string` objects as specified in the Framework Class Library.

Örnek - Şekiller



Örnek - Şekiller

- *TwoDimensionalShape* sınıfı ve *ThreeDimensionalShape* sınıfı tarafında kalıtılacak *Shape* soyut sınıfının oluşturulmalıdır.
- *Shape* soyut sınıfında
  - ilgili nesnenin ismini (Circle, Square vb..) döndürecek şekilde uygulanmış bir toString() metottu olmalıdır.
- Her bir alt sınıf için ilgili getArea() metotlar özelleştirilmelidir.
- *TwoDimensionalShape* sınıfında
  - getArea() metodu
- *ThreeDimensionalShape sınıfında*
  - *getArea() ve getVolume()* metotları olmalıdır.
- Bu özellikleri sağlayacak şekilde sınıf hiyerarşisi ve sınıf içleri doldurulmalı, Main() metodu bun göre dizayn edilmelidir.